

CS425- Coding Theory and Cryptography

Done by- 2203119, 2203121, 2203126

Github repository- 2203119: <https://github.com/MrunmayiKulkarni/ReedSolomanEncoder>

Github repository- 2203121: <https://github.com/nandeeshbhatrai/Reed-Solomon-Encoder>

Github repository- 2203126: <https://github.com/md-hzs-22/Reed-Solomon-encoder>

Report: REED-SOLOMON Encoding

Libraries/packages used:-

1. Numpy - For array and matrix manipulation
2. Tkinter - For GUI

Installation:-

Run: ``pip install numpy tkinter`` in the terminal to install these packages.

How to run:-

Run: ``python3 Reed-Solomon.py`` in the terminal to run the code.

Code Explanation:

I) Imports:

```
import tkinter as tk
from tkinter import messagebox
import numpy as np
```

II) Helper functions:

```
def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            return False
    return True
```

III) GUI

```
root = tk.Tk()
root.title("Reed-Solomon Encoder")

# Input n q k
tk.Label(root, text="n = q (Code length = prime):").grid(row=0, column=0)
get_n = tk.Entry(root)
get_n.grid(row=0, column=1)

tk.Label(root, text="k (Message length):").grid(row=1, column=0)
get_k = tk.Entry(root)
get_k.grid(row=1, column=1)

# Generate matrix
matrix_generate_button = tk.Button(root, text="Generate Parity Matrix", command=generate_parity_matrix)
matrix_generate_button.grid(row=2, columnspan=2)

# Parity matrix
matrix_frame = tk.Frame(root)
matrix_frame.grid(row=3, columnspan=2)

# Column swapper
swap_button = tk.Button(root, text="Swap and Encode", command=swap_columns)
swap_button.grid(row=4, columnspan=2)

tk.Label(root, text="Column 1:").grid(row=5, column=0)
col1_index = tk.Entry(root, width=5)
col1_index.grid(row=5, column=1)

tk.Label(root, text="Column 2:").grid(row=6, column=0)
col2_index = tk.Entry(root, width=5)
col2_index.grid(row=6, column=1)

# Message input
tk.Label(root, text="Message (space-separated values):").grid(row=7, column=0)
get_message = tk.Entry(root, width=30)
get_message.grid(row=7, column=1)

# Encode button
encode_button = tk.Button(root, text="Encode Message", command=encode_message)
encode_button.grid(row=8, columnspan=2)

# Encoded codeword
codeword_label = tk.Label(root, text="Encoded Codeword:")
codeword_label.grid(row=9, column=0)
codeword_display = tk.Label(root, text="", relief="solid", width=40)
codeword_display.grid(row=9, column=1)
```

First we create the main tkinter GUI frame using

```
root = tk.Tk()
```

We made 2 fields, one for getting $n(=q)$ and k which are length of the codeword and message respectively. Then we have added functionality to print the parity generator matrix and also the codeword for the given message. We have also added the facility to swap any columns in the PGM.

IV) PGM Generation algorithm:

```
alpha = [i for i in range(n)]

generated_matrix = []
for i in range(k):
    temp = []
    for j in range(n):
        temp.append(pow(alpha[j], i, q))
    generated_matrix.append(temp)

parity_matrix = np.array(generated_matrix) % q
```

V) Encoding algorithm:

```
encoded_word = np.dot(message, parity_matrix) % q
display_codeword(encoded_word)
```

The encoded codeword is the **dot product** of the **message** and the **PGM** adjusted with **modulo q**.